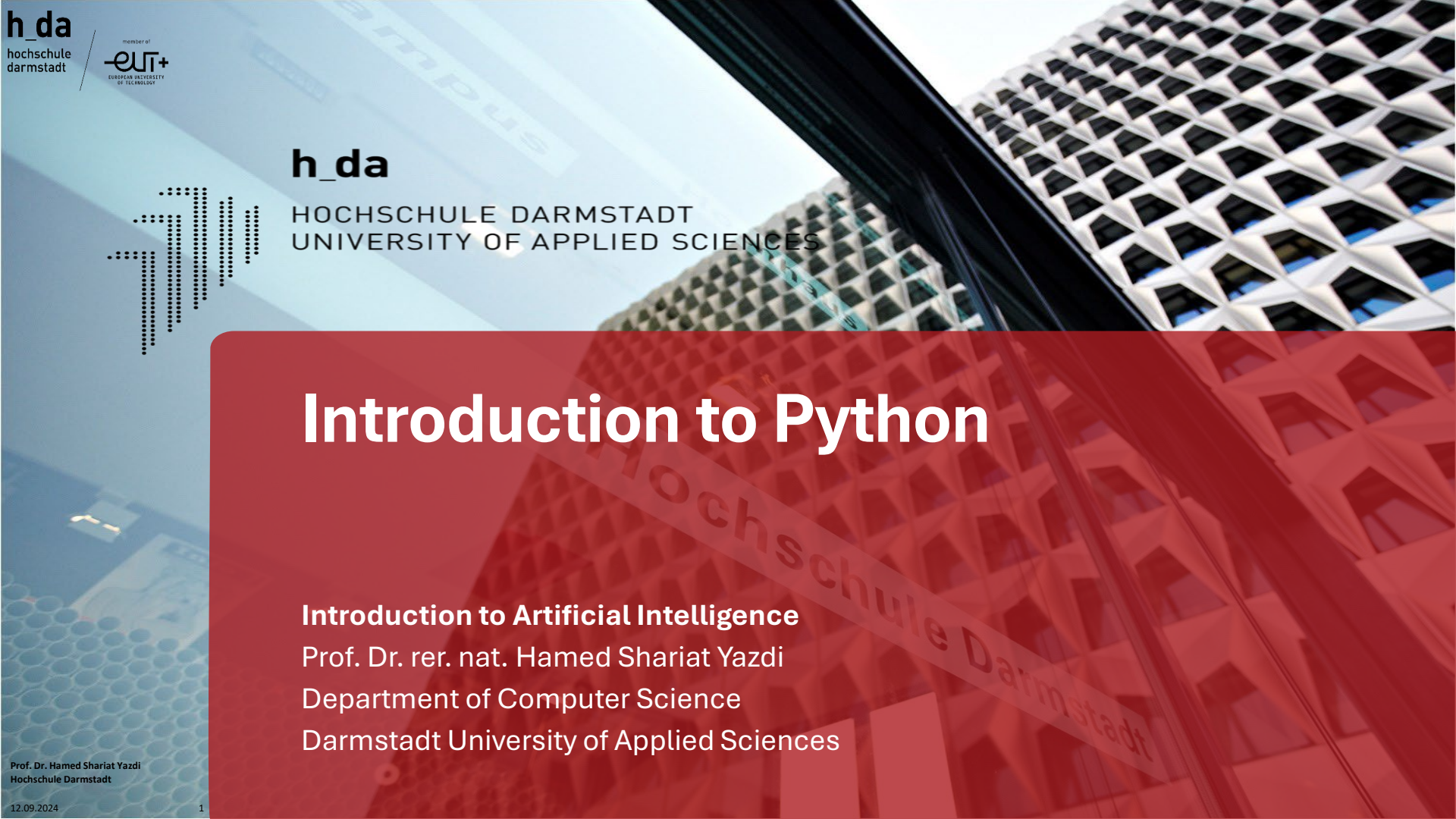

h_da
hochschule
darmstadt
member of
EUT+
EUROPEAN UNIVERSITY OF TECHNOLOGY
h_da
HOCHSCHULE DARMSTADT
UNIVERSITY OF APPLIED SCIENCES
Introduction to Python
Introduction to Artificial Intelligence
Prof. Dr. rer. nat. Hamed Shariat Yazdi
Department of Computer Science
Darmstadt University of Applied Sciences
Prof. Dr. Hamed Shariat Yazdi
Hochschule Darmstadt
12.09.2024
1

Agenda

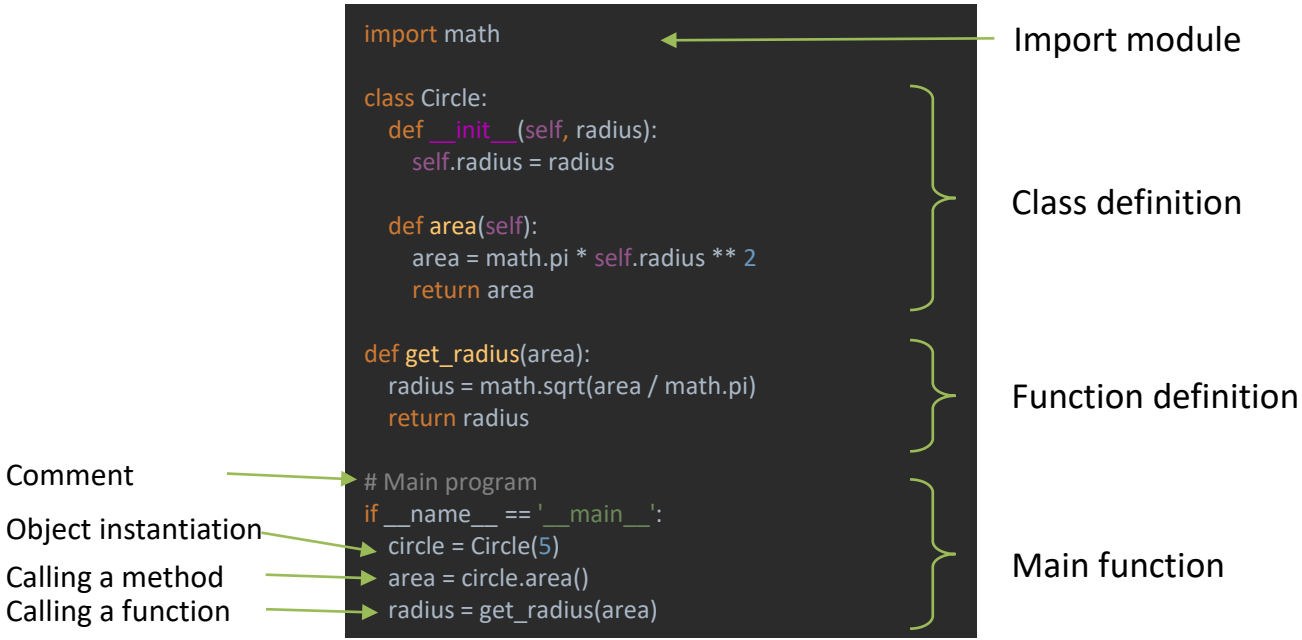
- **Introduction to Python**
- Basic Syntax
- Important structures
- Modules and packages
- Literature
- Quick Check

Prof. Dr. Hamed Shariat Yazdi
Department of Computer Science
Darmstadt Univeristy of Applied Sciences

Python

- Simple syntax – easy to learn *and* remember
  - No {} or () blocks – just indentation
- Efficient development – high productivity
- Interpreted (not compiled)
- Open source (essentially there is a package for any problem)
- Garbage collection
- Dynamic typing

Python - Example



Python – Example

```python
import math

class Circle:
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        area = math.pi * self.radius ** 2
        return area

def get_radius(area):
    radius = math.sqrt(area / math.pi)
    return radius

# Main program
if __name__ == '__main__':
    circle = Circle(5)
    area = circle.area()
    radius = get_radius(area)
```

Make code readable:

- Type hints
- Docstrings
- Line breaks

```python
import math

class Circle:
    def __init__(self, radius: float):
        """
        Initialize new circle of specified radius.

        :param radius: Radius of circle as float.
        """
        self.radius = radius

    def area(self) -> float:
        """
        Return area of circle.

        :return: Area of circle as float.
        """
        area = math.pi * self.radius ** 2
        return area

def get_radius(area: float) -> float:
    """
    Function to display the area of a given shape.

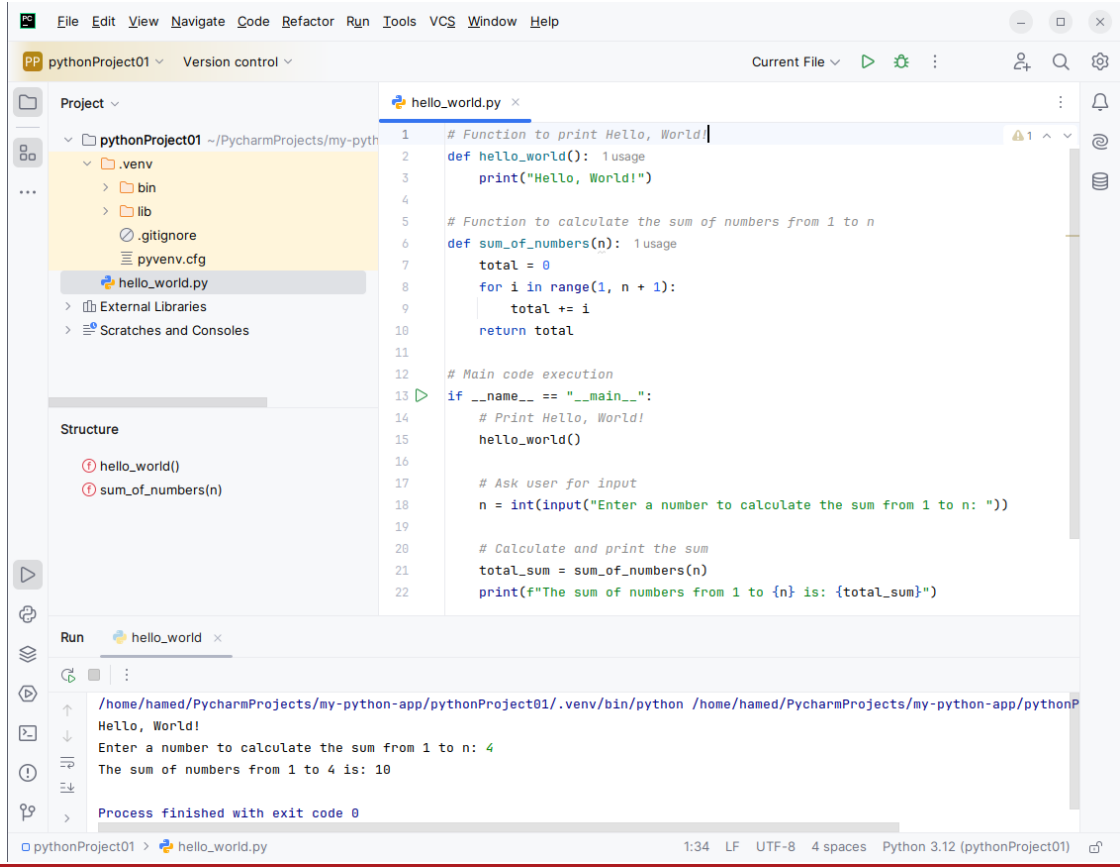
    :param area: Area of circle as float.
    :return: Radius of circle as float
    """
    radius = math.sqrt(area / math.pi)
    return radius


# Main program
if __name__ == '__main__':
    circle = Circle(5)
    area = circle.area()
    radius = get_radius(area)
```

Prof. Dr. Hamed Shariat Yazdi
Department of Computer Science
Darmstadt Univeristy of Applied Sciences

PyCharm (IDE)

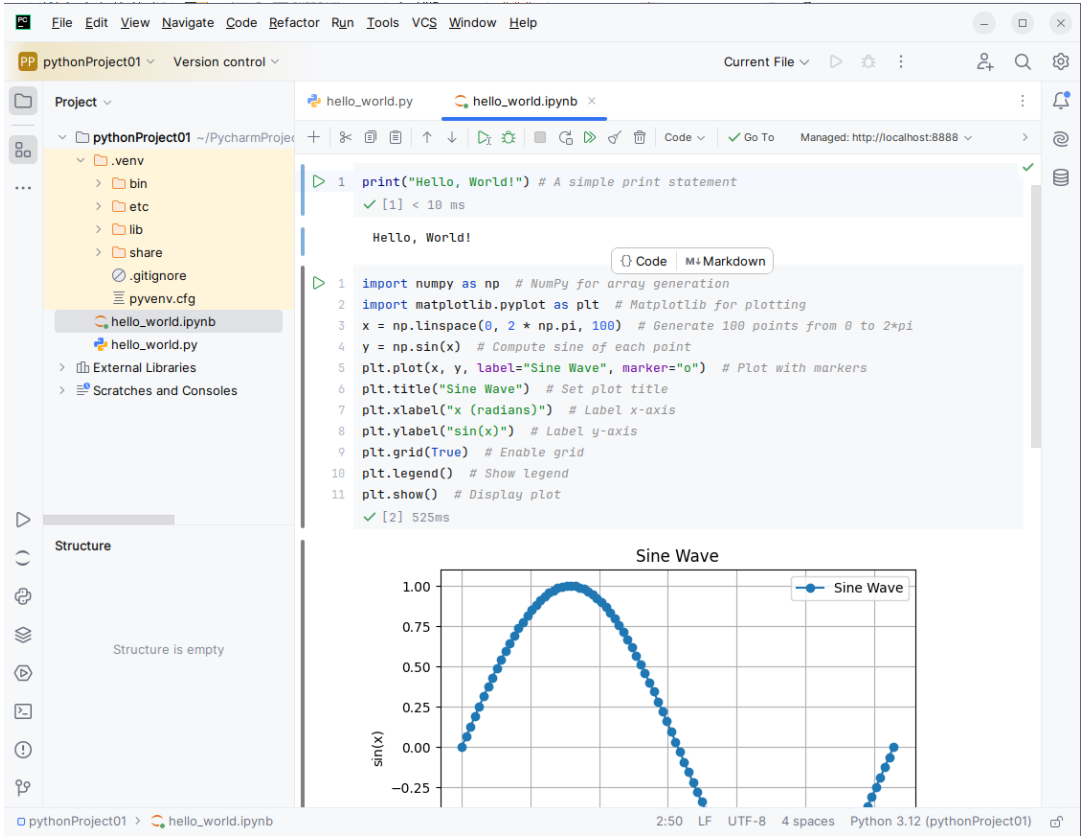
- Code completion and analysis
  - info when type hints do not match code
  - highlights style violations
- Code navigation (go to definition, find usage)
- Project management (and virtual environments)
- Cross-platform compatibility
- Version control integration
- Simple debugger
- **Setup:** see Installation Guide



Prof. Dr. Hamed Shariat Yazdi
Department of Computer Science
Darmstadt Univeristy of Applied Sciences

Jupyter Notebooks

- Mix of Python code and Markdown cells
  - Supports text, latex equations, images and more
- Interactive computation
  - One cell at a time
  - Real-time feedback
- Good for data exploration
  - Loads data into memory only once
  - Visualization etc.
- **Setup:** see Installation Guide



Prof. Dr. Hamed Shariat Yazdi
Department of Computer Science
Darmstadt Univeristy of Applied Sciences

Agenda

- Introduction to Python
- **Basic Syntax**
- Important structures
- Modules and packages
- Literature
- Quick Check

Basic Syntax

- See Quick Tutorial at
  - https://sites.cs.ucsb.edu/~tyang/class/240a16w/slides/pythonQuick.pdf
- Many built-in functions exist, for example

type() and isinstance()

```
In [1]: type("Hello")
Out[1]: str
In [2]: type(123)
Out[2]: int
In [3]: isinstance("Hello", str)
Out[3]: True
In [4]: isinstance(123, str)
Out[4]: False
```

Conversion functions

```
In [1]: int("123")
Out[1]: 123
In [2]: str(123)
Out[2]: '123'
In [3]: int(False)
Out[3]: 0
In [4]: bool(1)
Out[4]: True
```

Functions and methods of built-in data types

```
In [1]: x = [0, 1, 2, 3]
In [2]: len(x)
Out[2]: 4
In [3]: x.append(4)
In [4]: x
Out[4]: [0, 1, 2, 3, 4]
In [5]: x.extend([5, 6, 7])
In [6]: x
Out[6]: [0, 1, 2, 3, 4, 5, 6, 7]
```

Exercise

Write Python programs to solve the following tasks:

- Print “Hello, World!” to the screen.
- Generate and print a list of the squares of the numbers from 1 to 10.
- Calculate the sum of the even numbers between 1 and 50.
- Print a triangle pattern using asterisks (*) with a previously specified number of lines, e.g. for n = 5:



- Calculate and print the factorial of a given number, e.g. for n = 5 calculate

$$5! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1$$

Note: If you do not have Python installed, you can use an online Python compiler such as https://www.programiz.com/python-programming/online-compiler/ or https://www.online-python.com/

Agenda

- Introduction to Python
- Basic Syntax
- **Important structures**
- Modules and packages
- Literature
- Quick Check

Prof. Dr. Hamed Shariat Yazdi
Department of Computer Science
Darmstadt Univeristy of Applied Sciences

Dictionaries

- Useful to store (non-structured) information
- Similar to JSON format

```
x = {'german': 'Hallo', 'english': 'Hello', 'spanish': 'Hola'}
```

x['german']	'Hallo'
x['catalan']	KeyError: 'catalan'
x.get('german', 'Hey')	'Hallo'
x.get(catalan', 'Hey')	'Hey'
x.keys()	['german', 'english', 'spanish']
x.values()	['Hallo', 'Hello', 'Hola']
x.items()	[('german', 'Hallo'), ('english', 'Hello'), ('spanish', 'Hola')]

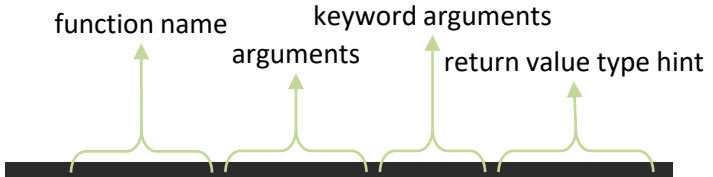
Functions

- Input: arguments (`args`) and keyword arguments (`kwargs`)
- Output: return value (default: None)
- Function block marked by indentation
- Might be assigned as value of a variable

```
function1 = summation
sum1 = function1(5, 6)
print(sum1)
>>> 11
```

- Anonymous functions using `lambda` expression

```
function2 = lambda a, b: a + b
sum1 = function2(5, 6)
print(sum1)
>>> 11
```




```
def summation(a: int, b: int, c: int = 0) -> int:
    """
    Sums two (or three) integers.

    :param a: First integer.
    :param b: Second integer.
    :param c: Third integer (optional). Defaults to 0.
    :return: Sum of the integers.
    """
    return a + b + c


if __name__ == '__main__':
    sum1 = summation(5, 6)
    sum2 = summation(5, 6, c=7)

    print(f"{sum1=}, {sum2=}")
```

Output

```
>>> sum1=11, sum2=18
```

Prof. Dr. Hamed Shariat Yazdi
Department of Computer Science
Darmstadt Univeristy of Applied Sciences

Exercise

- Write a function that takes as input two integers and an operator as string (`"+"`, `"-"`, `"*"`, `"/"`) and performs the desired calculation

```
def calculator(a: int, b: int, operator: str) -> float:
```

- Create a function that calculates the average grade of a student based on a dictionary of subject grades, e.g. {"Statistics": 85, "Math": 78, "IAI": 92}. Calculate and return the average grade of the student

```
def average_grade(grades: dict) -> float:
```

Note: If you do not have Python installed, you can use an online Python compiler such as https://www.programiz.com/python-programming/online-compiler/ or https://www.online-python.com/

Classes

- A class definition can contain various components
  - Class attributes
  - Instance attributes
  - Methods
  - Static methods
- The `__init__()` method defines what happens when an object is initialized

```
class Rectangle:
    version = "1.0.0"

    def __init__(self, height, width):
        self.height = height
        self.width = width

    def area(self):
        area = self.height * self.width
        return area

    @staticmethod
    def get_description():
        description = ("The area of a rectangle is the "
                       "product of its height and width")

        return description


if __name__ == '__main__':
    rect = Rectangle(3, 5)
    area = rect.area()
    description = rect.get_description()
    print(description)
    print(f"The rectangle's height is {rect.height}, its "
          f"width is {rect.width}, so its area is {area}")
```

Output

```
>>> The area of a rectangle is the product of its height and width
>>> The rectangle's height is 3, its width is 5, so its area is 15
```

Prof. Dr. Hamed Shariat Yazdi
Department of Computer Science
Darmstadt Univeristy of Applied Sciences

Class Inheritance

- Interfaces do not exist in Python
- Instead, we can use abstract base classes, that inherit from ABC
- Classes inheriting from abstract base classes must implement all abstract methods
- To override a method from a parent class, simply define it in child class
- To call a method from a parent class and add something, call it from the child class

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

class Rectangle(Shape):
    def __init__(self, height, width):
        self.height = height
        self.width = width

    def area(self):
        area = self.height * self.width
        return area

class Square(Rectangle):
    def __init__(self, side_length):
        super().__init__(side_length, side_length)

if __name__ == '__main__':
    square = Square(5)
    area = square.area()
    print(f"The square's height is {square.height}, its "
          f"width is {square.width}, so its area is {area}")
```

Output

```
>>> The square's height is 5, its width is 5, so its area is 25
```

Exercise

- Create a `BankAccount` class with attributes for the account holder's name and balance. Implement methods for depositing and withdrawing funds. Make sure that only money in the bank account can be withdrawn, so the balance cannot be negative. Create instances of this class and perform transactions.
- Write a `Student` class that has attributes for the student's name, age and grades (as dictionary, e.g. {"Statistics": 85, "Math": 78, "IAI": 92}). Implement a method to add subject grades to the dictionary. Create instances of the class and add grades to the students.

Note: If you do not have Python installed, you can use an online Python compiler such as https://www.programiz.com/python-programming/online-compiler/ or https://www.online-python.com/

Agenda

- Introduction to Python
- Basic Syntax
- Important structures
- **Modules and packages**
- Literature
- Quick Check

Prof. Dr. Hamed Shariat Yazdi
Department of Computer Science
Darmstadt Univeristy of Applied Sciences

Modules

- A module is a Python file (.py) containing definitions and statements
- We can use modules to separate code into different files and keep it clean
- Modules are imported with the `import` statement

```python
# rectangle.py
# ------------------------------------------------------------------
class Rectangle:
    def __init__(self, height, width):
        self.height = height
        self.width = width

    def area(self):
        area = self.height * self.width
        return area

if __name__ == '__main__':
    rect = Rectangle(3, 5)
    area = rect.area()
    print(f"The rectangle's height is {rect.height}, its "
          f"width is {rect.width}, so its area is {area}")
```

```python
# square.py
# ------------------------------------------------------------------
import rectangle

class Square(rectangle.Rectangle):
    def __init__(self, side_length):
        super().__init__(side_length, side_length)

if __name__ == '__main__':
    square = Square(5)
    area = square.area()
    print(f"The square's height is {square.height}, its "
          f"width is {square.width}, so its area is {area}")
```

Output

```
>>> The square's is Green and its area is 25
```

Prof. Dr. Hamed Shariat Yazdi
Department of Computer Science
Darmstadt Univeristy of Applied Sciences

Modules

- A module is a Python file (.py) containing definitions and statements
- We can use modules to separate code into different files and keep it clean
- Modules are imported with the `import` statement
- A single definition can be imported with the `from … import` statement
- Bad practice: use wildcards, e.g. `from rectangle import *`

```python
# rectangle.py
# ------------------------------------------------------------------
class Rectangle:
    def __init__(self, height, width):
        self.height = height
        self.width = width

    def area(self):
        area = self.height * self.width
        return area

if __name__ == '__main__':
    rect = Rectangle(3, 5)
    area = rect.area()
    print(f"The rectangle's height is {rect.height}, its "
          f"width is {rect.width}, so its area is {area}")
```

```python
# square.py
# ------------------------------------------------------------------
from rectangle import Rectangle

class Square(Rectangle):
    def __init__(self, side_length):
        super().__init__(side_length, side_length)

if __name__ == '__main__':
    square = Square(5)
    area = square.area()
    print(f"The square's height is {square.height}, its "
          f"width is {square.width}, so its area is {area}")
```

Prof. Dr. Hamed Shariat Yazdi
Department of Computer Science
Darmstadt Univeristy of Applied Sciences

Packages

- In larger projects, we can bundle multiple modules into packages
- A package is a directory containing an `__init__.py` file

```
project
├── __init__.py
├── colored_shape.py
├── shapes
│   ├── __init__.py
│   ├── shape.py
│   ├── rectangle.py
│   └── square.py
└── colors
    ├── __init__.py
    ├── red.py
    ├── green.py
    └── blue.py
```

```python
# colored_shape.py
# ----------------------------------------------------------------
from colors.green import Green
from shapes.square import Square

class ColoredShape:
    def __init__(self, color, shape):
        self.color = color
        self.shape = shape

    def area(self):
        return self.shape.area()

    def get_color(self):
        return self.color.get_color()

if __name__ == '__main__':
    square = Square(5)
    green = Green()
    green_square = ColoredShape(green, square)

    print(f"The square's is {green_square.get_color()} "
          f"and its area is {green_square.area()}")
```

Output

```
>>> The square's is Green and its area is 25
```

Prof. Dr. Hamed Shariat Yazdi
Department of Computer Science
Darmstadt Univeristy of Applied Sciences

We can import definitions in `__init__.py` to make it directly accessible

```
project
├── __init__.py
├── colored_shape.py
├── shapes
│   ├── __init__.py
│   ├── shape.py
│   ├── rectangle.py
│   └── square.py
└── colors
    ├── __init__.py
    ├── red.py
    ├── green.py
    └── blue.py
```

```python
# shapes/__init__.py
# ------------------------------------
from shape import Shape
from rectangle import Rectangle
from square import Square
```

```python
# colors/__init__.py
# ----------------------------
from red import Red
from green import Green
from blue import Blue
```

```python
# colored_shape.py
# ------------------------------------------------------------
from colors import Green
from shapes import Square

class ColoredShape:
    def __init__(self, color, shape):
        self.color = color
        self.shape = shape

    def area(self):
        return self.shape.area()

    def get_color(self):
        return self.color.get_color()

if __name__ == '__main__':
    square = Square(5)
    green = Green()
    green_square = ColoredShape(green, square)

    print(f"The square's is {green_square.get_color()} "
          f"and its area is {green_square.area()}")
```

Prof. Dr. Hamed Shariat Yazdi
Department of Computer Science
Darmstadt Univeristy of Applied Sciences

Packages

- Next to our own (internal) packages, we often use third-party (external) packages
- Common packages are
  - NumPy: efficient numerical computing
  - SciPy: scientific computing (based on NumPy)
  - Pandas: data manipulation and analysis
  - Matplotlib: static, animated and interactive plots
  - Scikit-learn: machine learning library
  - TensorFlow: deep learning library (Google)
  - PyTorch: deep learning library (Meta)
  - XGBoost: scalable and accurate gradient boosting



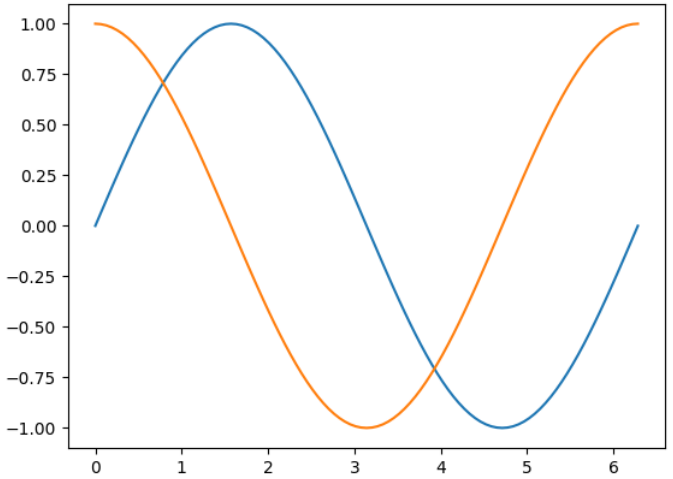
Image Source: Wikidata

Packages

```
import numpy as np
import matplotlib.pyplot as plt


if __name__ == '__main__':
    x = np.linspace(0, 2 * np.pi, 101)
    y = np.sin(x)
    z = np.cos(x)

    plt.plot(x, y)
    plt.plot(x, z)
    plt.show()
```



Exercise

Develop a movie catalog

- Create a module `movie.py` with a `Movie` class containing attributes like title, year and genre. Further implement a method to display this information.
- Create a module `catalog.py` with a `Catalog` class containing a list of movies. Implement methods to add movies and display to whole catalog.
- Finally, write a module `main.py` where you create instances of movies, add them to the catalog and display the entire catalog.

Note: If you do not have Python installed, you can use an online Python compiler such as https://www.programiz.com/python-programming/online-compiler/ or https://www.online-python.com/

Agenda

- Introduction to Python
- Basic Syntax
- Important structures
- Modules and packages
- **Literature**
- Quick Check

Literature

- Official Website: https://www.python.org/
- Documentation: https://docs.python.org/3/
- Beginner's Guide: https://wiki.python.org/moin/BeginnersGuide
- Tutorials:
  - python.org: https://docs.python.org/3/tutorial/index.html
  - Kaggle: https://www.kaggle.com/learn/python
  - W3 Schools: https://www.w3schools.com/python/
- PowerPoint slides (by Chaiporn Jaokaew and James Brucker, 2020): https://www.slideserve.com/anthonyahmed/introduction-to-the-python-language-powerpoint-ppt-presentation

Agenda

- Introduction to Python
- Basic Syntax
- Important structures
- Modules and packages
- Literature
- **Quick Check**

Quick Check

- How are code blocks marked in Python?
- What are some of the benefits of Python?
- How can we make Python code readable?
- What is the difference between a list and a tuple in Python?
- Which two types of arguments for functions exist?
- What is the difference between class attributes and instance attributes?
- What is the use of ABC (abstract base class) in Python?
- What is a Python module?
- Why do we use modules and packages?

Quick Check

- Is Python code compiled or interpreted?
- Are type hints mandatory?
- What are Python dictionaries?
- What is the difference between [1, 2, 3], (1, 2, 3) and {1, 2, 3} in Python?
- What is the lambda operator used for?
- What is the difference between a method and a static method in a Python class?
- What is a Python package?
- How can a module be imported in Python?
- How can a single definition be imported in Python?